

Sistema de Gerenciamento de Tarefas

MVP - Arquitetura de Software

Equipe:

Membro 1 – Eduardo Henrique

Membro 2 – José Guilherme

Membro 3 – Rochel Andrey

11 de Agosto de 2026

Sumário

1	Introdução	4
1.1	Contexto e Motivação	4
1.2	Objetivos	4
1.3	Tecnologias Utilizadas	4
2	Arquitetura do Sistema	4
2.1	Visão Geral da Arquitetura	4
2.2	Justificativa das Decisões Arquiteturais	5
2.3	Modelo C4	5
2.3.1	Nível 1: Contexto do Sistema	5
2.3.2	Nível 2: Containers	5
2.3.3	Nível 3: Componentes	6
2.3.4	Nível 4: Código	6
3	Implementação	6
3.1	Estrutura de Pastas	6
3.2	Model (Camada de Dados)	7
3.2.1	Conexão com o Banco de Dados	7
3.2.2	Operações CRUD	8
3.2.3	Validações	8
3.3	Controller (Server Actions)	8
3.4	View (Interface do Usuário)	9
3.4.1	Layout Global	9
3.4.2	Página de Listagem	9
3.4.3	Página de Detalhes	9
3.4.4	Página de Criação	9
3.4.5	Página de Edição	10
3.4.6	Componentes Reutilizáveis	10
4	Design Patterns	10
4.1	Factory Pattern	10
4.1.1	Contexto	10
4.1.2	Solução	10
4.1.3	Benefícios	11
4.2	Strategy Pattern	11
4.2.1	Contexto	11
4.2.2	Solução	11
4.2.3	Utilização	12
4.2.4	Benefícios	12
4.3	Observer Pattern	12
4.3.1	Contexto	12
4.3.2	Solução	12
4.3.3	Fluxo de Eventos	12
4.3.4	Benefícios	12
5	Documentação Arquitetural	13
5.1	Architecture Decision Records (ADRs)	13
5.2	C4 Model	13
5.2.1	Nível 1: Contexto do Sistema	13
5.2.2	Nível 2: Containers	13

6	Avaliação e Conclusão	14
6.1	Critérios de Projeto Atendidos	14
6.2	Desafios e Soluções	14
6.3	Conclusão	14
7	Demonstração do Sistema	14
8	Referências	17

1 Introdução

1.1 Contexto e Motivação

O presente projeto consiste no desenvolvimento de um Sistema de Gerenciamento de Tarefas (Kanban), concebido como MVP (Minimum Viable Product) para a disciplina de Arquitetura de Software. O sistema permite que usuários criem, listem, editem, visualizem detalhes e excluam tarefas, organizadas por status (A Fazer, Em Andamento, Concluído) e prioridade.

1.2 Objetivos

- Aplicar os princípios de projeto e padrões arquiteturais estudados ao longo da disciplina.
- Desenvolver um sistema funcional com arquitetura monolítica modular.
- Implementar três padrões de projeto: **Factory**, **Strategy** e **Observer**.
- Documentar as decisões arquiteturais utilizando *Architecture Decision Records* (ADRs).
- Modelar a arquitetura utilizando o modelo C4 (Contexto, Container, Componentes, Código).
- Adotar boas práticas de engenharia de software: alta coesão, baixo acoplamento e responsabilidade única.

1.3 Tecnologias Utilizadas

Tecnologia	Versão	Finalidade
Next.js	16.3.0	Framework fullstack (React + Node.js)
React	19.2.8	Biblioteca para interface do usuário
better-sqlite3	11.0.0	Banco de dados SQLite para persistência
GovBR-DS	3.7.0	Design System do Governo Federal Brasileiro
react-icons	5.7.0	Conjunto de ícones
PlantUML	-	Modelagem C4 Model

Tabela 1: Tecnologias e ferramentas utilizadas

2 Arquitetura do Sistema

2.1 Visão Geral da Arquitetura

O sistema adota uma **arquitetura monolítica modular**, construída sobre o framework Next.js no modelo App Router. A aplicação é organizada em três camadas principais, seguindo o padrão MVC (Model-View-Controller):

- **Model:** Camada de acesso a dados e regras de negócio, localizada em `lib/`.
- **View:** Componentes React e páginas que compõem a interface do usuário, localizados em `app/` e `components/`.
- **Controller:** Server Actions que orquestram as operações, localizadas em `app/tasks/actions.js`.

2.2 Justificativa das Decisões Arquiteturais

Decisão	Justificativa
Monolítico modular	Facilita a organização e separação de responsabilidades. Permite geração de HTML no servidor (SSR/SSG), eliminando a necessidade de API separada. Reduz complexidade de rede e acelera o desenvolvimento.
Next.js	Familiaridade da equipe com React e Node.js. Next.js oferece Server Components, Server Actions e API Routes, integrando perfeitamente front-end e back-end em um único projeto.
MVC	Padrão consolidado, simples e coeso. A separação entre Model, View e Controller facilita manutenção e testes.
SQLite	Ideal para MVP: configuração zero, arquivo único, portátil. Dispensa servidor de banco de dados dedicado.
GovBR-DS	Fornece componentes acessíveis e padronizados, garantindo consistência visual e usabilidade.

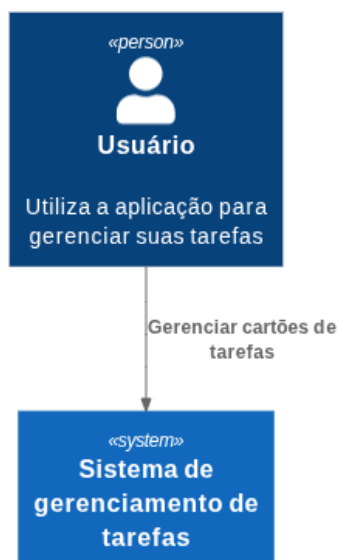
Tabela 2: Justificativa das decisões arquiteturais

2.3 Modelo C4

A arquitetura do sistema foi documentada utilizando o modelo C4, que fornece quatro níveis de abstração para compreensão do sistema.

2.3.1 Nível 1: Contexto do Sistema

O diagrama de contexto mostra a interação do sistema com o usuário final. O Sistema de Gerenciamento de Tarefas é representado como uma caixa central, e o ator "Usuário" interage com ele para gerenciar os cartões de tarefas.



2.3.2 Nível 2: Containers

O nível de containers detalha os principais blocos técnicos que compõem o sistema:

- **Web Application (Next.js):** Responsável pela interface do usuário e lógica de aplicação.
- **Database (SQLite):** Responsável pela persistência dos dados.

A comunicação entre o usuário e a aplicação web ocorre via HTTPS/TCP, e a aplicação web interage com o banco de dados via SQL/TCP.



2.3.3 Nível 3: Componentes

O nível de componentes detalha a estrutura interna do container **Web Application**. Os principais componentes são:

- **TaskListPage**: Exibe a lista de tarefas e gerencia a ordenação.
- **TaskForm**: Formulário para criação e edição de tarefas.
- **TaskCard**: Componente de exibição individual de uma tarefa.
- **StatusBadge**: Componente de exibição de status com cores.
- **Server Actions**: `createTaskAction`, `updateTaskAction`, `deleteTaskAction`, `listTasks`, `getTask`.
- **Model**: `task-model.js`, `task-factory.js`, `sort-strategies.js`, `event-bus.js`, `validators.js`.

2.3.4 Nível 4: Código

O nível de código foi abstraído em favor da documentação via ADRs e código autoexplicativo, seguindo o princípio de "documentação justa" (*just enough documentation*).

3 Implementação

3.1 Estrutura de Pastas

Listing 1: Estrutura do Projeto

```
/
app/
```

```

tasks/
  [id]/
    edit/
      page.js          # Edição
      page.js          # Detalhes
    new/
      page.js          # Criação
      actions.js       # Server Actions (Controller)
      page.js          # Listagem (View)
      layout.js        # Layout global
      not-found.js     # Página 404
      page.js          # Redireciona para /tasks
components/
  Header.jsx          # Cabeçalho
  StatusBadge.jsx     # Badge de status
  TaskCard.jsx        # Card de tarefa
  TaskForm.jsx        # Formulário reutilizável
lib/
  db.js               # Conexão SQLite
  event-bus.js        # EventEmitter (Observer)
  sort-strategies.js  # Estratégias de ordenação (Strategy)
  task-factory.js     # Fábrica de tarefas (Factory)
  task-model.js       # CRUD (Model)
  validators.js       # Validações
docs/
  adrs/               # Architecture Decision Records
    001-Adotar-arquitetura-monolitica.md
    002-Adotar-Framework-NextJs.md
  diagrams/
    c1_system_context.puml
    c1_system_context.png
    c2_container.puml
    c2_container.png
package.json
jsconfig.json

```

A estrutura de pastas segue a organização do Next.js App Router, separando claramente as responsabilidades: a pasta **app/** contém as rotas e páginas, **components/** agrupa os componentes reutilizáveis, **lib/** concentra a lógica de negócio e acesso a dados, e **docs/** armazena a documentação arquitetural.

3.2 Model (Camada de Dados)

A camada Model é responsável pelo acesso ao banco de dados, pela lógica de negócio, pela validação e pela criação de objetos. Ela é composta por quatro arquivos principais que trabalham em conjunto para garantir a integridade e consistência dos dados.

3.2.1 Conexão com o Banco de Dados

O arquivo **lib/db.js** é responsável por estabelecer e gerenciar a conexão com o banco de dados SQLite. Utilizando o pacote **better-sqlite3**, ele cria uma conexão síncrona e eficiente com o arquivo **tasks.db** localizado na raiz do projeto.

Durante a inicialização, o arquivo executa um comando **CREATE TABLE IF NOT EXISTS** para garantir que a tabela **tasks** exista. A tabela foi projetada com os seguintes campos: **id** como chave primária auto-incrementada, **title** como campo obrigatório, **description** opcional, **status** com valor padrão

'todo', `priority` com valor padrão 'medium', e os timestamps `created_at` e `updated_at` que são preenchidos automaticamente pelo SQLite.

A escolha do SQLite para um MVP é justificada pela simplicidade de configuração (arquivo único, sem servidor dedicado), portabilidade e adequação para a escala esperada do projeto.

3.2.2 Operações CRUD

O arquivo `lib/task-model.js` implementa as operações fundamentais de CRUD (Create, Read, Update, Delete) e atua como a camada de persistência do sistema. Cada função utiliza o objeto `db` exportado por `db.js` para executar comandos SQL preparados, garantindo segurança contra injeção de SQL.

A função `getAllTasks()` retorna todas as tarefas ordenadas por data de criação decrescente, exibindo primeiro as tarefas mais recentes. A função `getTaskById(id)` realiza uma busca específica por identificador, sendo utilizada nas páginas de detalhes e edição.

A operação de criação, `createTask(task)`, utiliza a `TaskFactory` (Factory Pattern) para normalizar os dados antes da inserção, garantindo valores padrão consistentes. Após a inserção, a função recupera a tarefa recém-criada e emite um evento `taskCreated` via `eventBus` (Observer Pattern), notificando os componentes interessados.

A atualização, `updateTask(id, updates)`, recebe um objeto com os campos a serem modificados, atualiza o registro no banco e atualiza automaticamente o campo `updated_at`. Após a atualização, emite o evento `taskUpdated`.

A exclusão, `deleteTask(id)`, remove a tarefa do banco e emite o evento `taskDeleted`, permitindo que a interface seja atualizada em tempo real sem recarregar a página.

3.2.3 Validações

O arquivo `lib/validators.js` implementa um conjunto de funções de validação que garantem a integridade dos dados antes de serem persistidos no banco. Cada função é especializada em um campo específico e lança erros descritivos quando a validação falha.

A função `validateTitle(title)` verifica se o título é uma string não vazia, com comprimento entre 3 e 100 caracteres, e retorna a versão sanitizada (trimada). A `validateDescription(description)` é opcional e permite até 500 caracteres, retornando uma string vazia para valores nulos.

As funções `validateStatus(status)` e `validatePriority(priority)` verificam se os valores estão entre os permitidos (`['todo', 'doing', 'done']` e `['low', 'medium', 'high']`, respectivamente) e normalizam para minúsculas.

A função `validateTask(task)` agrupa todas as validações individuais, retornando um objeto validado e sanitizado. Para operações de atualização parcial, a função `validateTaskPartial(updates)` valida apenas os campos fornecidos, ignorando os não informados.

Todas as validações são aplicadas pelas Server Actions antes de qualquer operação de escrita, garantindo que dados inválidos nunca cheguem ao banco.

3.3 Controller (Server Actions)

As Server Actions, localizadas em `app/tasks/actions.js`, atuam como controladores no padrão MVC, orquestrando a comunicação entre a View (interface do usuário) e o Model (camada de dados). Cada ação é uma função assíncrona executada no servidor, marcada com a diretiva `'use server'`.

As ações `listTasks()` e `getTask(id)` são wrappers simples que chamam diretamente as funções correspondentes do Model, servindo como pontos de entrada para a View obter dados.

As ações de escrita (`createTaskAction`, `updateTaskAction`, `deleteTaskAction`) seguem um padrão consistente: extraem os dados do `formData`, aplicam validações via `validateTask()`, executam a operação no Model e retornam um objeto com a estrutura `{ success: true, task: ... }` em caso de sucesso, ou `{ error: mensagem }` em caso de falha.

Todas as ações utilizam `revalidatePath('/tasks')` para invalidar o cache do Next.js, garantindo que as páginas sejam re-renderizadas com os dados mais recentes. Diferentemente de uma abordagem

tradicional com `redirect()`, as ações retornam o resultado para o cliente, permitindo que o frontend gerencie a navegação e emita eventos via Observer, mantendo a interface atualizada em tempo real.

3.4 View (Interface do Usuário)

A camada View é composta por componentes React e páginas do Next.js, organizadas de forma modular e reutilizável. Utiliza o GovBR Design System para estilização consistente e acessível.

3.4.1 Layout Global

O arquivo `app/layout.js` define o layout base de toda a aplicação. Ele importa o CSS do GovBR-DS, que fornece os estilos fundamentais para todos os componentes. Os metadados (`title` e `description`) são exportados para otimização de SEO e são automaticamente injetados pelo Next.js na tag `<head>`.

O layout estrutura a página com o componente **Header** no topo e uma área principal `<main>` com a classe `container` e padding vertical, garantindo consistência visual em todas as rotas. Por ser um Server Component (sem a diretiva `'use client'`), é executado no servidor para otimizar a renderização inicial.

3.4.2 Página de Listagem

A página de listagem (`app/tasks/page.js`) é um Client Component que gerencia o estado da lista de tarefas e as interações do usuário. Ela utiliza `useState` para armazenar as tarefas e a estratégia de ordenação selecionada, e `useEffect` para carregar os dados iniciais e configurar os listeners do Observer.

A função `loadTasks()` busca as tarefas via `listTasks()` e atualiza o estado, acionando a re-renderização da lista. O `useEffect` registra listeners para os eventos `taskCreated`, `taskUpdated` e `taskDeleted`, que recarregam a lista automaticamente sempre que uma tarefa é modificada (Observer Pattern).

A ordenação é aplicada dinamicamente utilizando o Strategy Pattern: o `strategyMap` associa cada opção do seletor a uma classe de estratégia, que é instanciada para ordenar a lista. Isso permite adicionar novas estratégias sem modificar o código existente.

3.4.3 Página de Detalhes

A página de detalhes (`app/tasks/[id]/page.js`) é um Server Component assíncrono que utiliza a API de parâmetros dinâmicos do Next.js 15+. A instrução `await params` desembrulha a Promise do parâmetro `id`, permitindo seu uso síncrono.

A página busca a tarefa via `getTask(id)` e, caso não exista, chama `notFound()` para exibir a página 404. O layout utiliza o sistema de grid do GovBR (`row, col-6`) para organizar as informações em duas colunas, e o componente **StatusBadge** para exibir o status com cores adequadas.

O botão de exclusão utiliza uma Server Action inline com importação dinâmica (`import()`), demonstrando uma abordagem flexível para ações que não precisam de estado local. Ao ser submetido, o formulário executa a ação no servidor e redireciona para a listagem.

3.4.4 Página de Criação

A página de criação (`app/tasks/new/page.js`) é um Client Component simples que renderiza o **TaskForm** com a flag `isEdit=false`. O formulário é responsável por coletar os dados do usuário e submetê-los à Server Action `createTaskAction`.

O botão "Cancelar" redireciona o usuário de volta para a listagem via `router.push('/tasks')`, sem perder os dados já inseridos (caso o usuário deseje cancelar a operação).

3.4.5 Página de Edição

A página de edição (`app/tasks/[id]/edit/page.js`) é um Client Component que carrega os dados da tarefa existente para pré-preenchimento do formulário. Utiliza `useParams()` para obter o ID da URL e `useState` para gerenciar o estado de carregamento.

O `useEffect` busca a tarefa via `getTask(id)` e atualiza o estado. Enquanto carrega, exibe uma mensagem de "Carregando...". Após o carregamento, renderiza o `TaskForm` com os dados da tarefa e a flag `isEdit=true`, permitindo que o usuário modifique os campos e submeta as alterações via `updateTaskAction`.

3.4.6 Componentes Reutilizáveis

Header O cabeçalho (`components/Header.jsx`) fornece navegação principal com links para a listagem e criação de tarefas. Utiliza ícones da biblioteca `react-icons` e classes do GovBR-DS para estilização, garantindo consistência com o restante da aplicação.

StatusBadge O componente `StatusBadge` (`components/StatusBadge.jsx`) é responsável por exibir o status da tarefa com cores específicas. Ele mapeia os valores do banco (`todo`, `doing`, `done`) para classes CSS do GovBR-DS (`warning`, `info`, `success`), e também traduz os valores para exibição em português (A Fazer, Em Andamento, Concluído). Isso garante uma interface amigável e consistente.

TaskCard O `TaskCard` (`components/TaskCard.jsx`) exibe as informações resumidas de uma tarefa e fornece botões de edição e exclusão. O título é um `Link` do Next.js que redireciona para a página de detalhes, enquanto os botões chamam funções recebidas via props (`onEdit` e `onDelete`), permitindo que o componente pai gerencie a lógica de navegação e exclusão.

TaskForm O `TaskForm` (`components/TaskForm.jsx`) é o componente mais complexo da camada View. Trata-se de um formulário reutilizável para criação e edição de tarefas, que utiliza `useActionState` (substituto do antigo `useFormState`) para gerenciar o estado da Server Action.

O componente mantém um estado local (`formData`) para feedback imediato ao usuário, sincronizado com os dados iniciais via `useEffect`. Ao submeter o formulário, a `formAction` executa a Server Action correspondente (criação ou atualização). Após o sucesso, o `useEffect` que monitora o estado `state` emite um evento via `eventBus` (Observer Pattern) e redireciona o usuário para a listagem.

O formulário também exibe mensagens de erro provenientes da Server Action, utilizando o componente `br-alert danger` do GovBR-DS, e desabilita os campos durante o processamento para evitar submissões duplicadas.

4 Design Patterns

Para atender aos requisitos do projeto, foram implementados três padrões de projeto (Design Patterns) que promovem flexibilidade, reutilização e baixo acoplamento.

4.1 Factory Pattern

4.1.1 Contexto

A criação de objetos `Task` estava dispersa pelo código, dificultando a manutenção e a garantia de valores padrão consistentes.

4.1.2 Solução

O padrão Factory foi implementado através da classe `TaskFactory`, que centraliza a criação de objetos tarefa com valores padrão.

```
export default class TaskFactory {
  static createTask(data = {}) {
    return {
      title: data.title || '',
      description: data.description || '',
      status: data.status || 'todo',
      priority: data.priority || 'medium',
      created_at: data.created_at || new Date().toISOString(),
      updated_at: data.updated_at || new Date().toISOString(),
    };
  }
}
```

4.1.3 Benefícios

- Centralização da lógica de criação.
- Garantia de valores padrão consistentes.
- Facilidade para extensão (novos campos, validações na criação).

4.2 Strategy Pattern

4.2.1 Contexto

A lista de tarefas precisa ser ordenada de diferentes formas (por data, título, status, prioridade). O acoplamento de múltiplos algoritmos em uma única função violaria o princípio Aberto/Fechado.

4.2.2 Solução

O padrão Strategy foi implementado através de classes de ordenação especializadas: `SortByDate`, `SortByTitle`, `SortByStatus`, `SortByPriority`. Cada classe implementa o método `sort(tasks)`.

```
export class SortByDate {
  sort(tasks) {
    return [...tasks].sort((a, b) => new Date(b.created_at) - new Date(a.created_at));
  }
}

export class SortByTitle {
  sort(tasks) {
    return [...tasks].sort((a, b) => a.title.localeCompare(b.title));
  }
}

export class SortByStatus {
  sort(tasks) {
    return [...tasks].sort((a, b) => a.status.localeCompare(b.status));
  }
}

export class SortByPriority {
  sort(tasks) {
    const ordem = { high: 1, medium: 2, low: 3 };
    return [...tasks].sort((a, b) => (ordem[a.priority] || 2) - (ordem[b.priority] || 2));
  }
}
```

```

    }
  }

export const strategyMap = {
  data: SortByDate,
  titulo: SortByTitle,
  status: SortByStatus,
  prioridade: SortByPriority,
};

```

4.2.3 Utilização

No `TaskListPage`, o usuário seleciona a estratégia desejada via `<select>`, e a classe correspondente é instanciada e utilizada para ordenar a lista.

4.2.4 Benefícios

- Separação dos algoritmos de ordenação.
- Facilidade para adicionar novas estratégias.
- Cumprimento do princípio Aberto/Fechado.

4.3 Observer Pattern

4.3.1 Contexto

A lista de tarefas precisa ser atualizada automaticamente sempre que uma tarefa é criada, atualizada ou excluída, sem necessidade de recarregar a página manualmente.

4.3.2 Solução

O padrão Observer foi implementado utilizando o `EventEmitter` nativo do Node.js, exportado como `eventBus`. O `eventBus` atua como o Subject, enquanto os componentes que emitem e escutam eventos atuam como Observers.

```

import EventEmitter from 'events';
const eventBus = new EventEmitter();
export default eventBus;

```

4.3.3 Fluxo de Eventos

1. **Emissão:** Quando uma ação é bem-sucedida, o componente emite um evento.
 - `TaskForm` emite `taskCreated` ou `taskUpdated`.
 - `TaskListPage` emite `taskDeleted` após exclusão.
2. **Recepção:** O `TaskListPage` escuta todos os eventos e recarrega a lista.

4.3.4 Benefícios

- Desacoplamento entre as operações e a atualização da lista.
- Atualização em tempo real sem recarregar a página.
- Facilidade para adicionar novos Observers.

5 Documentação Arquitetural

5.1 Architecture Decision Records (ADRs)

A documentação arquitetural foi realizada através de *Architecture Decision Records* (ADRs), seguindo o formato proposto por Michael Nygard.

Os documentos estão disponíveis no repositório

5.2 C4 Model

O sistema foi documentado utilizando o modelo C4, que fornece uma visão hierárquica da arquitetura em quatro níveis de abstração.

5.2.1 Nível 1: Contexto do Sistema

O diagrama de contexto mostra o sistema como uma caixa preta central, interagindo com o ator "Usuário".

```
@startuml C1 System Context
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Context.pu

Person(userPerson, "Usuário", "Utiliza a aplicação para gerenciar suas tarefas")

System(myTaskSystem, "Sistema de gerenciamento de tarefas")

Rel(userPerson, myTaskSystem, "Gerenciar cartões de tarefas")

@enduml
```

5.2.2 Nível 2: Containers

O diagrama de containers detalha a aplicação Next.js e o banco de dados SQLite, mostrando as tecnologias e protocolos de comunicação.

```
@startuml C2 Containers
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.

Person(userPerson, "Usuário", "Utiliza a aplicação para gerenciar suas tarefas")
System_Boundary(c1, "Sistema de gerenciamento de tarefas") {
    Container(web_app, "Web Application", "Next.js", "Permite que o usuário gerencie seus c
    ContainerDb(database, "Database", "SQLite", "Armezena dados da aplicação", "sqlite3")
}

Rel(userPerson, web_app, "Uses", "HTTPS/TCP")
Rel(web_app, database, "Ler/Escreve", "SQL/TCP")

@enduml
```

6 Avaliação e Conclusão

6.1 Critérios de Projeto Atendidos

Critério	Status
Monolítico modular	Atendido
MVC/CRUD	Atendido
Factory Pattern	Implementado em <code>lib/task-factory.js</code>
Strategy Pattern	Implementado em <code>lib/sort-strategies.js</code>
Observer Pattern	Implementado via <code>lib/event-bus.js</code>
Alta Coesão	Presente em componentes e módulos
Baixo Acoplamento	Mantido por interfaces e eventos
ADRs (mínimo 2)	2 ADRs documentados
C4 Model (Nível 1 e 2)	Diagramas PlantUML
Estilização GovBR-DS	Aplicada em todos os componentes

Tabela 3: Resumo dos critérios atendidos

6.2 Desafios e Soluções

- **Next.js 15 – Dynamic APIs:** O uso assíncrono de `params` foi ajustado com `await params`.
- **`useFormState` depreciado:** Migração para `useActionState` do React 19.
- **Observer entre servidor e cliente:** Ajuste para uso local do `eventBus`, com emissão de eventos no cliente após ações bem-sucedidas.

6.3 Conclusão

O MVP de Gerenciamento de Tarefas demonstra a aplicação prática dos princípios de arquitetura de software estudados na disciplina. A combinação de Next.js com arquitetura monolítica modular, MVC, Design Patterns (Factory, Strategy, Observer) e documentação via ADRs e C4 Model resultou em um sistema funcional, bem estruturado e preparado para evolução futura.

O projeto evidencia a importância de:

- Decisões arquiteturais baseadas em contexto e trade-offs.
- Separação de responsabilidades (Model, View, Controller).
- Uso de padrões de projeto para flexibilidade e reutilização.
- Documentação como ferramenta de comunicação e manutenção.

7 Demonstração do Sistema

Esta seção apresenta capturas de tela do sistema em funcionamento, demonstrando as principais funcionalidades implementadas.

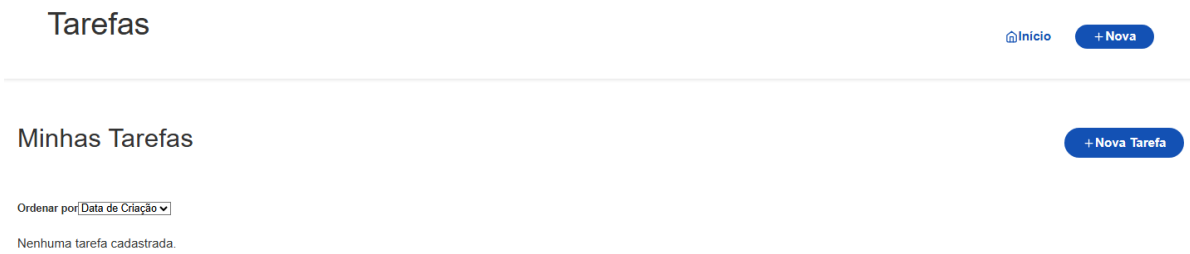


Figura 1: Página inicial com a lista de tarefas vazia.



Figura 2: Formulário de criação de uma nova tarefa, com campos de título, descrição e status.



Figura 3: Página de detalhes de uma tarefa, exibindo todas as informações e ações disponíveis.

Minhas Tarefas

[+ Nova Tarefa](#)

Ordenar por ▼ Data de Criação

Teste

Concluído

[✎ Editar](#)

[🗑 Excluir](#)

Teste

Em Andamento

[✎ Editar](#)

[🗑 Excluir](#)

Teste

A Fazer

[✎ Editar](#)

[🗑 Excluir](#)

N

Figura 4: Lista de tarefas com três tarefas em diferentes estados: A Fazer, Em Andamento e Concluído.

8 Referências

- VALENTE, Marco Tulio. **Engenharia de Software Moderna: Princípios e Práticas para Desenvolvimento de Software com Produtividade**. Edição do Autor, 2020.
- PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: Uma Abordagem Profissional**. 8. ed. Porto Alegre: AMGH/McGraw-Hill, 2016.
- GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Design Patterns: Elements of Reusable Object-Oriented Software**. Addison-Wesley, 1994.
- BROWN, Simon. **Software Architecture for Developers**. Leanpub, 2018.
- NYGARD, Michael. **Documenting Architecture Decisions**. Cognitect Blog, 2011. Disponível em: <https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: 11 ago. 2026.
- VERCEL. **Dynamic APIs are Asynchronous**. Next.js Documentation, 2024. Disponível em: <https://nextjs.org/docs/messages/sync-dynamic-apis>. Acesso em: 11 ago. 2026.